Chapter 1

Case Studies and Capstone Projects

1.1 Introduction

Every dataset used so far in this book, Cascade Home & Garden's satellite-derived car counts, Meridian Fabrication's financial statements, the ten-transaction fraud-scoring sample, was constructed for a single purpose: to make one method's mechanics checkable by hand, uncluttered by the messiness real data always carries. That was the right choice for teaching each method in isolation, but it leaves an honest gap this closing chapter exists to fill: a real financial dataset, of the kind a working data scientist actually downloads from a public repository such as the UCI Machine Learning Repository or Kaggle, arrives with undocumented category codes, an imbalanced target, correlated and redundant features, and no guarantee that the model that wins on one metric also wins on every other metric that matters.

This chapter works a single such dataset end to end, using nothing but tools this book has already built: feature engineering and a model bake-off spanning linear and tree-based methods, both from Chapter 6; real explainability tooling, Chapter 18's SHAP and permutation-importance methods, now run with the actual `shap` library rather than by hand; a cost-sensitive threshold choice extending Chapter 15's alert economics; and a real fairness audit extending Chapter 18's synthetic two-group example with genuine data. One topic here is genuinely new, calibration, and it is motivated directly by Chapter 9's own caveat that a credit score is only meaningful if the probability underneath it is accurate. The chapter then closes with governance considerations and a set of further capstone project ideas for the reader to pursue independently.

Two further, more advanced analyses push past what a first pass through this pipeline would typically include: a systematic hyperparameter search quantifying exactly how much is actually gained by tuning a model beyond a single, reasonable configuration, and an intersectional fairness cut that splits the test population two ways at once and uncovers a clear four-fifths-rule violation that this chapter's own single-attribute fairness audit, run first, entirely missed.

The dataset is the *Default of Credit Card Clients* dataset (Yeh and Lien, 2009), 30,000 Taiwanese credit card accounts with 23 raw features and a binary label, default on payment the following month, hosted on the UCI Machine Learning Repository and also widely distributed on Kaggle under the same name, the kind of source this chapter's title points to. Unlike every other dataset in this book, it was not built to illustrate anything in particular, and working through it surfaces problems no fictional dataset would: undocumented category codes that must be cleaned before any model can be fit, a real 22.1% default rate rather than a convenient round number, and, as this chapter's fairness audits show directly, a model that passes a single-attribute fairness check comfortably while failing an intersectional one, even though no protected attribute is among its inputs.

This chapter's structure mirrors the order a working data scientist actually follows on a new dataset, and is worth previewing before working through the details. It begins where any real project begins, inspecting the raw data closely enough to find its undocumented quirks (Section 1.2.1) and engineering features from raw monthly statements rather than starting from an already-clean feature table (Section 1.2.2). It then compares several model families on identical, held-out data (Section 1.3.1) and checks whether the winning model's probabilities can actually be trusted at face value (Section 1.3.2). From there it explains that model's behavior using real software rather than hand computation (Section 1.3.3), converts model output

Wulin.Suo@Queensu.ca

into an actual lending decision under an explicit cost assumption (Section 1.3.4), and audits that decision for fairness, first along one dimension (Section 1.3.5) and then, in Section 1.4.2, along two at once. A closing discussion of what deploying this pipeline in production would actually require (Section 1.4.1) is followed by an explicit, reusable template distilled from that same pipeline (Section 1.4.3), and then by a set of further capstone projects to apply the template to (Section 1.4.4). Those end the chapter, and the book, on the same note the introduction to Chapter 1 opened it: these tools are only as trustworthy as the discipline applied in using them.

1.2 The Data

The capstone begins where every real project does: with a dataset that is not what its documentation claims.

1.2.1 The Capstone Dataset and Its Imperfections

The raw dataset records, for each of 30,000 accounts, a credit limit (`LIMIT_BAL`), demographic fields (`SEX`, `EDUCATION`, `MARRIAGE`, `AGE`), six months of repayment status codes (`PAY_1` through `PAY_6`, where -1 indicates payment made duly and a positive integer indicates the number of months a payment is overdue), six months of billed amounts (`BILL_AMT1`–`BILL_AMT6`), six months of amounts actually paid (`PAY_AMT1`–`PAY_AMT6`), and the target, default on the following month's payment.

Loading the data immediately surfaces the first real-world imperfection the fictional datasets of earlier chapters never had: the data dictionary documents `EDUCATION` as taking values 1 (graduate school), 2 (university), 3 (high school), or 4 (other), yet the actual column also contains 14 rows coded 0, 280 rows coded 5, and 51 rows coded 6, none of which the documentation defines. `MARRIAGE` shows the same pattern: documented as 1 (married), 2 (single), or 3 (other), the column also contains 54 rows coded 0.

Neither anomaly is disclosed anywhere in the dataset's documentation, and a modeler who does not notice them will silently feed a machine learning model category codes with no defined meaning. Following the standard practice for this well-studied dataset, both sets of undocumented codes are folded into their respective "other" categories (0, 5, and 6 into `EDUCATION`'s existing category 4; 0 into `MARRIAGE`'s existing category 3) before any modeling begins, a cleaning step with no equivalent anywhere else in this book precisely because every other dataset was built clean by construction.

The repayment-status columns carry two further quirks of the same kind. First, the raw file has no `PAY_1` at all: the most recent month's status column is named `PAY_0`, while the billed and paid amounts run from `BILL_AMT1` and `PAY_AMT1`, so the companion notebook renames `PAY_0` to `PAY_1` to line the three six-month series up. Second, the dictionary defines the status codes as -1 (paid duly) and 1 through 9 (months overdue) only, yet the columns are dominated by two undocumented codes, -2 and 0, which together make up 66.9% of all monthly status entries and appear at least once for 25,939 of the 30,000 accounts. They are commonly read as "no balance to pay" and "revolving credit, minimum payment made," but nothing in the official documentation says so. This chapter leaves them as they are; since both rank below 1, neither counts as a delinquency in the `max_delinquency` feature defined below, which is a modeling decision a reader should notice rather than inherit silently.

Table 1.1 summarizes four representative raw fields before any cleaning or feature engineering, and is itself worth inspecting closely rather than skimming past, the habit Chapter 6 recommended before fitting any model to any dataset.

Field	Mean	Std. dev.	Min	Median	Max
`LIMIT_BAL` (NT$)	167,484	129,748	10,000	140,000	1,000,000
`AGE` (years)	35.5	9.2	21	34	79
`BILL_AMT1` (NT$)	51,223	73,636	−165,580	22,382	964,511
`PAY_AMT1` (NT$)	5,664	16,563	0	2,100	873,552

Table 1.1: Summary statistics for four raw fields (before cleaning)

`BILL_AMT1`'s negative minimum is yet another genuine data quirk beyond the code anomalies above, not

Wulin.Suo@Queensu.ca

documented anywhere in the dataset's description: a negative billed amount means the account was in credit, having overpaid the previous month, rather than owing anything at all, a legitimate state for a revolving credit account but one that a modeler unfamiliar with the data might mistake for a data-entry error and drop or clip without justification. `PAY_AMT1`'s heavy right skew, a median of just NT$2,100 against a mean of NT$5,664 and a maximum of NT$873,552, is the kind of distribution Section 1.2.2's engineered ratio features are designed to normalize before they reach a linear model, though the tree-based models in Section 1.3.1 are considerably less sensitive to this kind of skew than logistic regression is, since a tree splits on a feature's rank order rather than its raw scale.

Two further properties of the raw data are worth noting before any model is fit. The first is the class balance. A 22.1% default rate is considerably higher than a typical, real-world consumer credit-card portfolio, which more commonly runs in the low single digits, and the gap is a reminder that a published research dataset's class balance reflects however it happened to be sampled and whichever period it happened to be drawn from, not necessarily the balance any other institution's own portfolio will show. This data covers a six-month window in 2005, shortly before a Taiwanese consumer-credit crisis. A model built on it should therefore not be assumed to transfer its absolute predicted probabilities to a materially different portfolio without recalibration, even if its ranking of relatively risky against relatively safe accounts transfers reasonably well.

The second property is redundancy. Several of the raw features are highly correlated with each other by construction: the six `BILL_AMT` columns track a single account's revolving balance from one month to the next, so they move together closely. That matters directly for Section 1.3.3's explanation exercise, because an importance or attribution score assigned to any one of several mutually correlated features is, to some extent, arbitrary, a model being able to substitute one for another with little effect on its predictions. Chapter 6 raised this issue in the abstract when discussing multicollinearity in linear regression, and it carries over, in a different technical form, to tree-based models and their explanations alike.

1.2.2 From Raw Fields to Model-Ready Features

Chapter 6 introduced feature engineering in the abstract; this section performs it on genuinely raw fields for the first time. Five engineered features summarize the six months of repayment and billing history into a form a model can use more directly than six separate monthly columns each. `max_delinquency` is the worst (maximum) repayment-status code across all six months, a single number capturing whether an account was ever seriously overdue even if its most recent month looks clean. `avg_utilization` is the average, across the six months, of billed amount divided by credit limit, a standard credit-risk signal Chapter 9 introduced conceptually but never computed from raw statement data. `total_bill` and `total_paid` are the six-month sums of billed and paid amounts. And `pay_to_bill_ratio` is the ratio of the two, capturing how much of what an account owed it actually paid back over the window; it is clipped at 5 to bound the influence of a small number of accounts with an unusually large payment relative to a near-zero bill. These five, together with 20 of the 23 raw fields, give 25 features in total. The three raw fields left out are `SEX`, `MARRIAGE`, and `AGE`: taking U.S. fair-lending law as the reference point, as Chapter 18 did, the Equal Credit Opportunity Act and its Regulation B forbid basing a credit decision on sex or marital status and allow age only in narrow ways (for instance, as a variable in a demonstrably and statistically sound scoring system that never treats applicants aged 62 or older less favorably), so the cleanest choice is to keep all three out of the model and hold `SEX` and `AGE` aside purely as attributes for Sections 1.3.5 and 1.4.2's fairness audits.

The 30,000 accounts are split 80/20 into a training set of 24,000 and a held-out test set of 6,000, stratified by the default label so that both sets preserve the same 22.1% default rate, using a fixed random seed so the split (and every result in this chapter) is exactly reproducible from the companion notebook. Every model comparison, calibration check, explanation, threshold choice, and fairness audit in the rest of this chapter is computed on this same held-out test set, never on the data any model was fit to. Chapter 6 introduced that discipline as essential to detecting overfitting, and it matters more, not less, once a real dataset's quirks make it easy to convince oneself that a model has learned something it has only memorized.

One further discipline is worth naming explicitly because a real dataset makes it easy to violate by accident: every one of this section's engineered features, `max_delinquency` included, is computed from information available at the time each account's repayment history was recorded, not from anything that depends on knowing whether the account actually defaulted. This sounds obvious, but Chapter 17's point-

Wulin.Suo@Queensu.ca

in-time discipline for alternative data applies here in a more subtle form: a feature engineered carelessly from a dataset that also contains the outcome column, for instance a rolling average that accidentally incorporates a future month's billing data, can leak information about the label into the features and produce a model that looks excellent on a held-out test set drawn from the same leaking dataset, yet fails immediately once deployed on genuinely new applicants for whom that same leakage is not available. This dataset's structure happens to make such leakage easy to avoid, since every raw field precedes the single target month by construction, but the discipline of checking for it explicitly, not merely assuming a clean-looking dataset has none, is the habit a reader should carry into Section 1.4.4's further projects, several of which (the Lending Club dataset in particular) are considerably easier to leak by accident than this one.

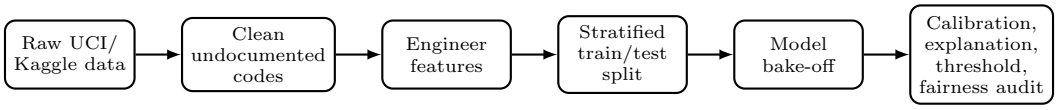


Figure 1.1: The end-to-end pipeline this chapter works through by hand

1.3 Modeling and Evaluation

This part runs a full bake-off, then asks the questions that decide whether any of it is usable.

1.3.1 A Model Bake-Off: Logistic Regression, Random Forests, and Gradient Boosting

Four models are fit on the training set and evaluated, once, on the untouched test set: a plain logistic regression (Chapter 6's workhorse, standardized features, no adjustment for class imbalance), a class-weighted logistic regression (Chapter 15's first remedy for imbalanced classification, applied here directly rather than to a stylized example), a random forest (300 trees, Chapter 6's bagging-based ensemble), and gradient-boosted trees via XGBoost (300 boosting rounds, Chapter 6's sequential, bias-reducing ensemble family, named but not fit by hand in earlier chapters).

Table 1.2 reports seven metrics for each.

Model	Accuracy	Precision	Recall	F_1	ROC-AUC	PR-AUC	Brier
Logistic (plain)	0.807	0.683	0.238	0.353	0.720	0.494	0.146
Logistic (balanced)	0.731	0.426	0.615	0.503	0.721	0.485	0.205
Random Forest	0.816	0.660	0.350	0.458	0.775	0.554	0.136
XGBoost	0.818	0.665	0.356	0.464	0.778	0.557	0.135

Table 1.2: Held-out test set performance for four models on the credit-default dataset

The plain logistic regression's accuracy (80.7%) looks respectable in isolation, the trap Chapter 15 warned against for a rare-event problem: with a 22.1% default rate, a model that is far too conservative about flagging defaults can still post high accuracy simply by defaulting, so to speak, to predicting the majority class, and this model's recall of only 23.8% shows it is missing more than three-quarters of the accounts that actually default at the standard 0.5 threshold. Class weighting flips the trade-off almost exactly as Chapter 15 described in the abstract: recall rises to 61.5% while precision falls from 68.3% to 42.6%, a substantially better F_1 (0.503 versus 0.353) but, notably, a distinctly worse Brier score (0.205 versus 0.146). That pairing is the first concrete evidence in this book that class weighting is not a free improvement. Reweighting the training loss to counter class imbalance also distorts the model's output away from a well-calibrated probability estimate, a real cost Chapter 15 never had occasion to quantify with an imbalance-sensitive metric like the Brier score, since its own worked example used a perfectly balanced confusion-matrix illustration rather than a held-out probability calibration check.

Wulin.Suo@Queensu.ca

The two tree ensembles dominate both discrimination metrics, ROC-AUC and PR-AUC, over either logistic regression. The random forest versus XGBoost comparison also gives Chapter 6's abstract bagging-versus-boosting distinction a concrete empirical instance for the first time: XGBoost's sequential, error-correcting boosting edges out the random forest's parallel, variance-reducing bagging on every metric in this table, though by a narrower margin (ROC-AUC 0.778 versus 0.775) than the gap either ensemble opens up over logistic regression. That pattern is consistent with a general finding, well documented in the applied machine learning literature, that gradient boosting and random forests are far closer competitors with each other on tabular data than either is with a linear model, even though boosting frequently comes out slightly ahead in practice.

This comparison is also a useful corrective to a common oversimplification, that a single logistic regression is always the "safe, interpretable" choice and a tree ensemble is always the "powerful, opaque" one. Both ensembles here outperform both logistic regressions on every ranking-based metric by a wide margin, roughly five to six points of ROC-AUC, while remaining explainable in the sense Chapter 18 formalized, using the real tooling Section 1.3.3 applies below, just not in the closed-form, coefficient-reading sense that made Chapter 9's linear credit-scoring model interpretable by construction.

A lender choosing between these four models is therefore not simply trading accuracy for interpretability along a single dial, as Chapter 6's abstract discussion sometimes suggests; it is trading a small amount of post-hoc explanation overhead (Section 1.3.3's SHAP computation) for a real, measurable gain in both discrimination and, as Section 1.3.2 shows next, calibration.

1.3.2 Is the Model's Probability Actually a Probability? Calibration in Practice

Chapter 9 noted, in passing, that a credit score is only meaningful to the extent that the default probability underlying it is itself well calibrated, not merely useful for ranking applicants from safest to riskiest; this section makes that check concrete for the first time.

Calibration asks whether, among all accounts a model assigns a predicted default probability of, say, 30%, roughly 30% actually default. Table 1.3 bins the test set into ten deciles of predicted probability for the plain logistic regression and for XGBoost, and compares each decile's average predicted probability to its actual observed default rate.

	Logistic (plain)		XGBoost	
Decile (low to high)	Mean predicted	Mean actual	Mean predicted	Mean actual
1	0.040	0.135	0.036	0.045
5	0.167	0.118	0.131	0.152
6	0.185	0.120	0.159	0.183
10	0.577	0.670	0.725	0.697

Table 1.3: Calibration by predicted-probability decile: plain logistic regression versus XGBoost

XGBoost's predicted and actual rates track closely, within three percentage points, in every decile shown (and, checking the full ten-bin table in the companion notebook, in every decile not shown here as well), consistent with its lower Brier score in Table 1.2. The plain logistic regression shows a more visible gap in its lowest decile, an average predicted probability of 4.0% against an actual default rate of 13.5%, more than three times higher, meaning the safest-looking accounts by this model's own ranking are considerably riskier in reality than the model's probabilities suggest, the kind of miscalibration a lender relying on this model's raw probability, rather than just its rank ordering, would want caught before deployment.

This is precisely why Section 1.3.4's cost-sensitive threshold choice and Section 1.3.5's fairness audit are both computed using XGBoost's probabilities rather than the plain logistic regression's, since both exercises depend on the model's predicted probability meaning what it claims to mean, not merely on the model's ability to rank risky accounts above safe ones.

Wulin.Suo@Queensu.ca

Fixing Miscalibration: Isotonic Recalibration

Identifying the plain logistic regression's calibration gap raises the natural next question: can it be corrected without discarding the underlying model and its accuracy? **Isotonic recalibration** fits a separate, nonparametric, monotonically increasing function that remaps a model's raw predicted probabilities onto better-calibrated ones. The remapping is learned from data the original model was fit on, but evaluated out-of-fold, which is Section 1.2.2's train/test discipline applied one level deeper, to the recalibration step itself. Fitting the remapping function directly on the test set, the way a careless analyst might, rather than by cross-validation on the training set alone, would leak the very data this recalibration is meant to be validated against.

Applied to the plain logistic regression, scikit-learn's `CalibratedClassifierCV` with isotonic regression, five-fold cross-validated entirely within the training set, improves the test-set Brier score from 0.1460 to 0.1419, a modest but genuine reduction, while leaving ROC-AUC essentially unchanged (0.720 before, 0.724 after). That is exactly what an isotonic remapping should do. Because the transformation is monotonic, it cannot change how the model ranks any two applicants relative to each other, only how well its stated probability matches reality at a given rank. The improvement is concentrated exactly where Table 1.3 found the original problem.

Recomputing the reliability table on the recalibrated probabilities for the lowest-probability bin,

$$\begin{aligned} \text{Recalibrated:} &\quad \text{predicted} = 9.8\%,\ \text{actual} = 12.3\%,\ \text{ratio} \approx 1.2, \\ \text{Original (uncalibrated):} &\quad \text{predicted} = 4.0\%,\ \text{actual} = 13.5\%,\ \text{ratio} > 3, \end{aligned}$$

This is a genuinely useful, general lesson beyond this one dataset: a model with strong discriminative power but poor calibration, the plain logistic regression's situation here, does not need to be replaced to become trustworthy as a probability estimate, only recalibrated, a considerably cheaper fix in practice than retraining an entirely different model family, and one that leaves every one of the model's original coefficients, and therefore its adverse-action interpretability, completely intact.

It is worth being equally clear about what recalibration does not fix: XGBoost's own Brier score (0.135) remains lower than the recalibrated logistic regression's (0.142) even after this correction, since XGBoost's advantage was never purely a calibration artifact to begin with, its higher ROC-AUC in Table 1.2 reflects genuinely superior discriminative power that a monotonic remapping of the logistic regression's probabilities cannot manufacture out of a linear model's more limited functional form.

Calibration matters beyond this chapter's own threshold and fairness exercises, too. A bank reporting expected credit losses under accounting standards such as IFRS 9 or the analogous U.S. CECL framework multiplies a probability of default directly into a provisioning calculation, Chapter 9's own EL = PD × LGD × EAD decomposition. A systematically miscalibrated probability, not merely a poorly ranked one, therefore translates directly into an over- or understated loss provision on the bank's own financial statements. That provision is exactly the kind of number Chapter 4 taught how to read and Chapter 9 taught how to model, and it is shown here to depend on a property, calibration, that a model's accuracy or even its ROC-AUC does not by itself guarantee.

Section 1.3.3 returns to this formula with real provisioning numbers, once this chapter's own low-risk and high-risk applicants have been introduced.

1.3.3 Explaining a Real, Nonlinear Model: SHAP and Permutation Importance

Chapter 18 computed an exact Shapley decomposition by hand for Chapter 9's credit-scoring model only because that model was linear in the log-odds; XGBoost is not, so the same exact closed form is unavailable here. The `shap` library's `TreeExplainer` algorithm computes exact Shapley values efficiently for tree ensembles specifically, without requiring the linear-model shortcut Chapter 18 relied on, by exploiting the tree structure directly rather than approximating with sampled perturbations the way Chapter 18's LIME example did.

Table 1.4 compares the resulting global feature ranking, mean absolute SHAP value across 1,000 held-out accounts, against Chapter 18's other global method, permutation importance (here, the drop in ROC-AUC when a feature is shuffled), computed on the full test set.

Wulin.Suo@Queensu.ca

Rank	Permutation importance (feature)	Mean |SHAP| (feature)
1	`PAY_1` (0.0337)	`max_delinquency` (0.444)
2	`max_delinquency` (0.0318)	`PAY_1` (0.339)
3	`avg_utilization` (0.0118)	`avg_utilization` (0.167)
4	`BILL_AMT1` (0.0094)	`BILL_AMT1` (0.130)
5	`LIMIT_BAL` (0.0057)	`LIMIT_BAL` (0.118)

Table 1.4: Top features by two global importance methods, XGBoost model

The two methods agree closely on which five features matter, the most recent repayment status, the worst repayment status over six months, utilization, the most recent bill, and credit limit, and even order ranks three through five identically, but disagree on the ranking of the top two: permutation importance ranks the single most recent month's repayment status (`PAY_1`) above the worst-ever status across six months, while SHAP ranks them the other way around. Neither ranking is simply wrong; they answer different questions, the caution Chapter 18 raised about permutation importance describing a feature's contribution to a specific performance metric on specific data, while SHAP describes each feature's average marginal contribution to individual predictions, and a modeler who reports only one of the two risks overstating a false precision about which single feature "matters most."

A single applicant's explanation makes this concrete at the individual level Section 1.3.1's aggregate table cannot. The first test-set applicant is assigned a predicted default probability of 13.07% by XGBoost.

SHAP's additive decomposition operates on the model's raw log-odds output rather than on the probability directly, exactly as Chapter 18's exact Shapley decomposition operated on log-odds for the linear credit-scoring model. This applicant's base value, the average log-odds prediction across the training population, is -1.272, and summing the individual SHAP contributions from every feature gives a total log-odds of -1.895. The sigmoid function converts that back to 13.07%, exactly matching the model's actual output. This verifies the same completeness property Chapter 18 checked by hand for a linear model, now confirmed for a genuinely nonlinear ensemble instead. The applicant's largest contributions are

$$\begin{aligned}
\texttt{max_delinquency} = 0 \text{ (no serious delinquency):} &\quad \text{SHAP} = -0.309,\\
\texttt{avg_utilization} = 12.1\% \text{ (low):} &\quad \text{SHAP} = -0.215,\\
\text{Credit limit} = \text{NT\$50,000 (modest):} &\quad \text{SHAP} = +0.175,\\
\text{Most-recent repayment status} = -1 \text{ (paid duly):} &\quad \text{SHAP} = -0.151,
\end{aligned}$$

the modest credit limit contributing positively to predicted risk since, in this dataset, lower limits are associated with higher-risk accounts.

Unlike Chapter 18's adverse-action reason codes, which followed directly and automatically from an exactly linear model's coefficients, producing this same kind of individualized explanation for a real deployed ensemble requires this extra step, running `TreeExplainer` against the fitted model, before a bank could issue a genuine, model-derived reason to an applicant under the Equal Credit Opportunity Act.

A contrasting, genuinely high-risk applicant from the same test sample makes the method's value clearer still. This second applicant is assigned a predicted default probability of 92.2%, and the same decomposition gives a base value of -1.272 and a summed log-odds of 2.470 (sigmoid of which is, again exactly, 92.2%). Here the dominant contributions run in the opposite direction from the low-risk applicant above:

$$\begin{aligned}
\text{Most-recent repayment status} = 3 \text{ (three months overdue):} &\quad \text{SHAP} = +1.009,\\
\text{Worst-ever repayment status} = 7 \text{ (severely delinquent):} &\quad \text{SHAP} = +0.921,\\
\text{Credit limit} = \text{NT\$10,000 (the dataset's minimum):} &\quad \text{SHAP} = +0.303,
\end{aligned}$$

with `pay_to_bill_ratio` $= 0$ (repaid essentially nothing over six months) adding a further $+0.151$, and the most-recent-repayment contribution the single largest seen in either applicant's decomposition.

Placed side by side, the two applicants' explanations read almost as mirror images, no delinquency history and full repayment pushing one applicant's risk sharply down, recent and severe delinquency alongside near-zero repayment pushing the other's sharply up, the kind of concrete, individualized contrast an adverse-action

Wulin.Suo@Queensu.ca

notice or an internal credit-committee review would need, and exactly what a single aggregate feature-importance ranking like Table 1.4 cannot provide on its own.

A Worked Example: From Predicted Probability to a Provisioning Number

Section 1.3.2 promised to return to Chapter 9's EL = PD × LGD × EAD decomposition with real numbers; the two applicants just explained supply exactly what is needed. Each applicant's credit limit stands in for exposure at default, a standard simplifying assumption for a revolving credit-card line assumed fully drawn at the moment of default, and, absent any collateral information for an unsecured card balance, this example reuses Chapter 9's own illustrative LGD = 60%.

All amounts in this dataset are in New Taiwan dollars (NT$), not U.S. dollars; at the 2005 exchange rate of roughly NT$32 to the U.S. dollar, the low-risk applicant's NT$50,000 limit is worth only about US$1,550. For the low-risk applicant, PD = 13.07077% (the 13.07% reported earlier, carried to more decimals to reproduce the figure below exactly by hand) and EAD = NT$50,000 (this applicant's `LIMIT_BAL`), giving

$$\text{EL}_{\text{low-risk}} = 0.1307077 \times 0.60 \times \text{NT\$50,000} = \text{NT\$3,921.23},$$

while the high-risk applicant, with PD = 92.19913% (similarly carried to more decimals) and a much smaller EAD = NT$10,000 (a lower credit limit, itself consistent with this applicant's weaker profile), gives

$$\text{EL}_{\text{high-risk}} = 0.9219913 \times 0.60 \times \text{NT\$10,000} = \text{NT\$5,531.95}.$$

The high-risk applicant's expected loss is only about 1.41× the low-risk applicant's, a far smaller multiple than the two applicants' 7.05× gap in predicted default probability alone (92.20%/13.07%), because the high-risk applicant's credit limit, one-fifth of the other's, largely offsets an otherwise much larger provisioning number, the kind of interaction between the three separate inputs to expected loss that looking at PD in isolation would miss. A bank provisioning against its entire card portfolio would sum a version of this calculation across every account, so a systematic miscalibration of the kind Section 1.3.2 found and fixed in the plain logistic regression, understating risk in the model's lowest-probability decile, would understate the provisioning total specifically for the bank's safest-looking accounts, understating reserves in the segment a supervisor examining the bank's provisioning adequacy would be least likely to scrutinize closely.

`TreeExplainer`'s speed is itself worth noting: it computes exact Shapley values for a tree ensemble in time roughly proportional to the number of trees times the square of tree depth, rather than the exponential cost the general, model-agnostic Shapley formula Chapter 18 mentioned in passing would require, which is precisely why it is practical to run it here against a genuine, several-hundred-tree ensemble rather than only against the small, linear, two-feature toy models Chapter 18's hand computations were limited to. This matters operationally, not just theoretically: a bank issuing thousands of adverse-action notices a month needs an explanation method fast enough to run at that volume, and a sampling-based method like Chapter 18's LIME, however useful for a single, illustrative explanation, does not scale to that volume nearly as cleanly as an algorithm purpose-built for the specific model family being explained.

1.3.4 Choosing an Operating Threshold: A Real Cost-Sensitive Analysis

Every result so far used the default 0.5 classification threshold; Chapter 15's alert-economics section argued that the right threshold is a business decision balancing the cost of a missed default against the cost of denying a good customer, not a property of the model itself, and this section quantifies that trade-off directly rather than only in the abstract. Assigning an illustrative cost of $500 to a missed default (a false negative, an account that defaults but is approved) and $50 to wrongly denying a good customer (a false positive, a tenth of the missed-default cost, reflecting that a denied good customer typically costs a lender foregone interest income rather than a lost principal balance), the expected total cost across the test set is $\text{Cost}(t) = 500 \cdot FN(t) + 50 \cdot FP(t)$ at a given threshold t.

Table 1.5 sweeps this across a grid of thresholds using XGBoost's predicted probabilities.

The cost-minimizing threshold on this grid is $t = 0.10$, far below the naive 0.5 default, at a total expected cost of $209,750, versus $438,900 at the naive threshold, more than double. That value could have been

Wulin.Suo@Queensu.ca

Threshold	False negatives	False positives	Total cost
0.05	29	4,071	$218,050
0.10 (cost-minimizing)	129	2,905	$209,750
0.20	450	1,218	$285,900
0.35	667	510	$359,000
0.50 (naive default)	854	238	$438,900
0.60	918	179	$467,950

Table 1.5: Cost-sensitive threshold sweep, XGBoost model, held-out test set

predicted without any sweep. Denying an account with predicted default probability p is cheaper than approving it whenever $p \times \$500 > (1-p) \times \$50$, so for a calibrated model the cost-minimizing threshold is

$$t^* = \frac{c_{FP}}{c_{FP} + c_{FN}} = \frac{50}{50 + 500} = 0.091,$$

the same expected-cost comparison Chapter 15 used to derive its fraud-review threshold, adapted to a decision in which a correctly denied defaulter costs nothing. Only the ratio of the two costs matters, not their level or currency. The grid's empirical minimum at 0.10 sits right next to this analytic value (at exactly 0.091 the test-set cost is $214,100, the small difference being sampling noise in 6,000 accounts), which is itself a further check on Section 1.3.2's finding that XGBoost's probabilities are well calibrated.

The gap between $209,750 and $438,900 is itself the lesson: choosing a classification threshold by convention rather than by the institution's actual cost structure leaves real money on the table, considerably more here than the gap between any two models in Table 1.2, which is why Chapter 15 treated threshold choice as at least as consequential as model choice. The specific optimal threshold found here depends entirely on the assumed 10-to-1 cost ratio; a lender who instead estimated missed defaults and denied good customers as comparably costly would land on a threshold much closer to 0.5, underscoring that this is a genuine business judgment, not a number this chapter's methods can supply on their own.

The operating point this produces deserves an equally plain statement: at $t = 0.10$ the model approves only 31.6% of the test accounts, against 88.2% at the naive 0.5 threshold, and turns away 62.2% of the accounts that would not have defaulted. No real card lender operates anywhere near that strict a cutoff. The toy cost ratio is what drives it: charging a missed default only ten times a wrongly denied good customer, against a 22.1% base default rate, pushes the threshold below the probability most accounts are assigned, whereas a real lender's cost of denying a good customer includes the lifetime value of the relationship, and the cost of a missed default is proportional to the exposure rather than a flat amount, so a realistic cost model would place the threshold, and the approval rate, very differently.

This threshold sweep is, in effect, a business-cost-weighted walk along the same receiver operating characteristic curve Chapter 6 introduced in the abstract. Every threshold in Table 1.5 corresponds to exactly one point on that curve, trading a lower false-negative count for a higher false-positive count as the threshold falls. What this section adds is not a new statistical concept but a conversion of that familiar trade-off directly into dollars, so that the operating point is chosen by comparing two cost figures rather than by squinting at a curve and picking whichever point looks like a reasonable compromise.

A lender in practice would also want to stress-test this choice the way Chapter 9 stress-tested a credit portfolio: if a recession doubled the true cost of a missed default relative to a denied applicant, the cost-minimizing threshold would fall further still, flagging even more accounts as high-risk, the kind of scenario analysis a model-risk function should run before, not after, committing to a single operating threshold in production.

1.3.5 A Real Fairness Audit

Chapter 18's fairness audit used a synthetic eight-applicant dataset built to illustrate a point cleanly; this section runs the identical audit, the four-fifths rule and an error-rate comparison, on the actual test set,

Wulin.Suo@Queensu.ca

at the cost-minimizing threshold from Section 1.3.4, split by the dataset's `SEX` field. Table 1.6 reports the results.

Group	n	Approval rate	False-negative rate	False-positive rate
Male	2,402	30.9%	8.9%	62.5%
Female	3,598	32.1%	10.3%	62.0%

Table 1.6: A fairness audit by sex, XGBoost model at the cost-minimizing threshold ($t = 0.10$)

The adverse impact ratio, computed from the unrounded approval rates as 30.85%/32.13% = 0.960, clears the 80% four-fifths-rule threshold Chapter 18's fairness section formalized by a wide margin, and some gap is to be expected here, since male accounts in the test set actually default more often (23.4% against 21.3%). The error-rate comparison shows the same tension Chapter 18's synthetic example illustrated, now with genuine data: the group with the higher approval rate (female accounts) also has the higher false-negative rate, missing 10.3% of accounts that actually go on to default, versus 8.9% for male accounts, so the group nominally favored by the headline approval-rate statistic is simultaneously the group whose real default risk this specific threshold captures somewhat less completely. Neither the approval-rate comparison nor the error-rate comparison is "the" fairness result; both are properties of the same model and the same threshold, and, exactly as Chapter 18's fairness-impossibility result predicts, no adjustment to this one threshold alone can bring both gaps to zero simultaneously unless the two groups' underlying default rates happen to coincide.

It is worth being explicit about what this audit does, and does not, establish. Leaving the protected attributes out of the model mattered: the same XGBoost configuration refit with `SEX`, `MARRIAGE`, and `AGE` as inputs, which the companion notebook runs purely for contrast, approves 28.6% of male and 33.9% of female accounts at the same threshold, an adverse impact ratio of only 0.843, while scoring essentially no better (test ROC-AUC 0.779 against 0.778). But removing the attributes did not remove the difference. The model never sees `SEX`, and yet its decisions still differ measurably by sex, the pattern Chapter 18's Apple Card case vignette described in a real, consumer-facing product: "fairness through unawareness" narrows a disparity only as far as the remaining features fail to encode the omitted attribute. What this audit does not establish, on its own, is why the residual difference exists: whether it reflects the groups' genuinely different default rates or some other feature correlated with sex closely enough to reproduce the pattern. Income proxies and credit-limit assignment history are the leading suspects in this dataset, though testing that hypothesis rigorously would require the causal counterfactual-fairness framing Chapter 18's advanced topics introduced, not merely the correlational check performed here. Section 1.4.2 then shows directly that this single-attribute finding, reassuring as its 0.960 ratio looks in isolation, does not remotely tell the whole story once a second characteristic is added to the audit.

1.4 From Model to Project

A model is not a project; this part covers deployment, and the template for doing all of it again.

1.4.1 From Model to Deployment: Governance Considerations

Chapter 18's governance section organized model governance around SR 11-7's three pillars, conceptual soundness, ongoing monitoring, and outcomes analysis, and this pipeline maps onto all three concretely rather than abstractly.

Conceptual soundness is Section 1.3.1's model bake-off and Section 1.3.2's calibration check, documented well enough that an independent validator, Chapter 15's second line of defense, could reproduce every number in Table 1.2 from the companion notebook without needing to re-derive the modeling choices from scratch. Outcomes analysis is Section 1.3.5's fairness audit, which this chapter's own numbers show passing comfortably on one cut (0.960 by sex) and failing on another (Section 1.4.2's sex-by-age cut), so a one-time check at deployment is not sufficient; ongoing monitoring, using Chapter 18's population stability index

Wulin.Suo@Queensu.ca

against this exact feature set and a recurring rerun of both audits, is what would catch a shift in the applicant pool moving either ratio before an examiner does.

None of this replaces human judgment: Section 1.3.4's cost assumptions, the $500-to-$50 ratio driving the entire threshold choice, are a business decision no algorithm in this chapter can make on the institution's behalf, and choosing them is the kind of accountable, documented judgment call Chapter 15's three-lines-of-defense framework and the NIST AI RMF's "govern" function both exist to make explicit rather than implicit.

A concrete deployment checklist follows directly from this chapter's own sections, and is worth stating explicitly rather than leaving implicit. Before this model ever scores a real applicant, an institution would need Section 1.2.1's cleaning logic and Section 1.2.2's feature engineering encoded in a reproducible data pipeline rather than a one-off notebook cell. It would need Section 1.3.1's model selection and Section 1.4.2's hyperparameter search documented well enough that a validator could rerun them independently and obtain the same numbers reported here. It would need Section 1.3.2's calibration check and Section 1.3.4's cost assumptions re-approved on a recurring basis rather than fixed once at launch, since a model's calibration and an institution's true cost ratio both drift as the underlying applicant population and business environment change. And it would need Section 1.3.5 and Section 1.4.2's fairness audits, the intersectional cut included, rerun on every new cohort of applicants rather than merely at initial deployment, precisely because Section 1.4.2 showed directly that a single-attribute check passing today is no guarantee that a deeper cut would not fail tomorrow.

1.4.2 Advanced Topics

Two further analyses push this chapter's pipeline beyond what a first pass through it would typically include: a systematic search over XGBoost's hyperparameters, to see how much Section 1.3.1's single, reasonably chosen configuration actually left on the table, and a fairness audit that splits the test set two ways at once rather than one, revealing a disparity Section 1.3.5's single-attribute audit could not see at all.

Hyperparameter Tuning: Diminishing Returns in Practice

Section 1.3.1's XGBoost model used a single, manually chosen configuration (300 trees, maximum depth 4, learning rate 0.05). A randomized search over 30 candidate configurations varied tree depth (3 to 6), learning rate (0.01 to 0.1), the number of boosting rounds (100 to 500), the row and column subsampling fractions, and the minimum child weight controlling how conservatively a tree can split. Every candidate was evaluated by five-fold cross-validation on the training set alone, never touching the test set. That is exactly Section 1.2.2's train/test discipline applied one level deeper, since even a hyperparameter search must be validated on data the search itself never saw.

The best configuration found, depth 4, learning rate 0.1, 100 rounds, no row subsampling, 60% column subsampling, and a minimum child weight of 5, achieved a cross-validated ROC-AUC of 0.786, and, refit on the full training set and evaluated once on the held-out test set, a test ROC-AUC of 0.781, a PR-AUC of 0.554, and a Brier score of 0.135. Compared against Section 1.3.1's untuned XGBoost model (test ROC-AUC 0.778, PR-AUC 0.557, Brier 0.135), the result of an actual, reasonably thorough hyperparameter search is not even a uniform improvement: ROC-AUC rises by three thousandths, PR-AUC falls by three thousandths, and the Brier score does not move at the third decimal, all changes far smaller than the gap either tree ensemble opened up over logistic regression in Table 1.2.

This is a genuinely useful, if less exciting, finding. Once a model family is well chosen and its configuration is set to sensible, moderate values, as Section 1.3.1's untuned model already was, further hyperparameter search on a dataset of this size and feature set faces diminishing returns. The modest gap between the cross-validated score (0.786) and the held-out test score (0.781) carries its own small, honest reminder: a hyperparameter search's selected score is mildly optimistic, having been chosen, across 30 candidates, precisely because it happened to perform best on that particular cross-validation split.

Intersectional Fairness: What Aggregation Can Hide

Section 1.3.5's audit split the test set only by sex. Splitting it a second way at the same time, by sex and by an age cutoff at 30 years, reveals a pattern the single-attribute audit could not, shown in Table 1.7.

Wulin.Suo@Queensu.ca

Sex	Age group	n	Approval rate	False-negative rate
Male	Under 30	669	23.9%	4.7%
Male	30 and over	1,733	33.5%	10.5%
Female	Under 30	1,243	30.0%	10.1%
Female	30 and over	2,355	33.3%	10.4%

Table 1.7: An intersectional fairness cut: sex crossed with age group

The most disadvantaged subgroup by approval rate, young men under 30, is approved only 23.9% of the time, against 33.5% for the most favored subgroup, men 30 and over. That is an adverse impact ratio, from the unrounded rates, of 23.92%/33.53% = 0.713, a clear four-fifths-rule violation, and it stands in sharp contrast to Section 1.3.5's sex-only audit, which found a ratio of 0.960, comfortably on the right side of the same threshold. The two findings are not in tension; they are simply answers to different questions. Averaging young and older men together in the sex-only audit obscures the fact that young men, considered on their own, face a starkly lower approval rate than any other subgroup in this cross-tabulation, a disparity the single-attribute view cannot see because it never asks the question in the first place. Nor is this simply an age effect that a separate age audit would have caught at the same strength: among women the under-30 group is approved at 30.0% against 33.3%, a ratio of about 0.90, so the shortfall is concentrated where the two characteristics meet. It also arises in a model that, as Section 1.2.2 specified, never sees `AGE` at all, a second and sharper instance of Section 1.3.5's point that leaving an attribute out does not keep the features that correlate with it, shorter credit histories and lower assigned limits among the youngest borrowers, from carrying it back in; the most obvious such feature here is `LIMIT_BAL`, since the youngest borrowers, and young men above all, hold markedly lower credit limits (a median of NT$80,000 for men under 30 in the test set, against NT$150,000 for men 30 and over). This is precisely the failure mode fair-lending examiners test for directly under the doctrine of intersectional or subgroup analysis: a model can pass every single-attribute audit an institution runs and still produce a stark disparity for a subgroup defined by two or more characteristics at once, and the only way to find such a disparity is to look for it specifically, since Table 1.6's aggregate numbers give no indication whatsoever that it might exist.

A genuinely thorough fairness audit therefore cannot stop at Section 1.3.5's single-attribute cut, however clean its result looked; it must deliberately cross every combination of characteristics an institution has reason to worry about, a substantially larger testing burden than a single audit table might suggest, and one this dataset's own numbers show is not merely a theoretical concern.

1.4.3 A Template for Completing a Capstone Project

Section 1.4.4 below lists several real datasets a reader could build a capstone project on. This section is the template for actually doing so, an explicit, stage-by-stage checklist distilled directly from this chapter's own pipeline (Figure 1.1), so that a reader starting from any dataset on that list, or from one of their own choosing entirely, has a concrete standard to work toward rather than an open-ended instruction to "build a model." Each stage in Table 1.8 names the specific tool this chapter, or an earlier one, developed for it, along with a deliverable, a specific artifact, table, or number, that the stage should produce before the project moves on to the next. A project that cannot produce every deliverable in the table is not yet finished, whatever its headline accuracy number looks like.

A project report built around this template naturally falls into five sections, which a reader can use directly as a report or presentation outline. The **executive summary** states Stage 1's problem statement and the headline result in a single paragraph a non-technical reader could act on. **Data and methodology** covers Stages 2 and 3, written so that another analyst could reproduce the cleaning, the feature engineering, and the model choices exactly. **Results** presents Stages 3 and 4's tables together, so that a reader can judge discrimination and calibration side by side rather than being shown only whichever number happens to look most favorable. **Explainability and fairness** covers Stages 5 through 7, which belong together: an explanation of what drives the model's decisions is also evidence for or against a fairness concern, exactly as Section 1.3.5's discussion of `LIMIT_BAL` and Section 1.4.2's intersectional finding both showed directly. **Limitations and recommendations** carries Stage 8's governance memo, together with an explicit, named

Wulin.Suo@Queensu.ca

Stage	What this stage requires	Deliverable to produce and report
1. Problem framing and dataset selection	State the business decision the model will inform in one sentence (approve or deny a loan, flag or clear a transaction, and so on); confirm the chosen dataset has a genuine, non-leaking target label and enough rows to support a genuinely held-out test set.	A one-paragraph problem statement naming the target variable, its base rate, and the decision the model's output will feed into.
2. Data cleaning and feature engineering	Inspect every column's summary statistics and value counts before fitting anything, exactly Section 1.2.1's method; document any undocumented or anomalous codes found and how each was resolved; engineer at least one derived feature not present in the raw columns, and explicitly check it (and every other feature) for information that would not genuinely have been available at the moment a real prediction would be made, Section 1.2.2's leakage discipline.	A short data-quality memo listing every anomaly found and the cleaning decision made, plus the engineered feature list with a one-sentence leakage justification for each.
3. Train/test discipline and model bake-off	A stratified train/test split with a fixed, reported random seed; at least one intrinsically interpretable model (linear or logistic regression) and at least one tree ensemble, evaluated once, on the held-out test set alone, on accuracy, precision, recall, F_1, ROC-AUC, PR-AUC, and Brier score, Section 1.3.1's method.	A results table in the same format as Table 1.2, plus both sets' target base rates confirming the stratification held.
4. Calibration check	A reliability table (deciles of predicted probability against actual outcome rate) for at least the best-discriminating model, Section 1.3.2's method.	A calibration table in the format of Table 1.3, plus one sentence stating whether the model's raw probability, not merely its ranking, can be trusted at face value.
5. Explainability	A global importance ranking (permutation importance or mean absolute SHAP value) and at least one individual, local explanation for a specific prediction, Section 1.3.3's method.	A top-five global feature ranking and one specific case's local explanation, with its completeness check (SHAP values plus base value reproducing the model's actual output) shown explicitly.
6. Threshold and cost analysis	An explicit, stated assumption for the dollar cost of a false negative and a false positive, and the resulting cost-minimizing threshold found by sweeping a grid of candidate thresholds, Section 1.3.4's method.	The two assumed costs, the resulting threshold, and its total expected cost compared explicitly against the naive 0.5 threshold's cost.
7. Fairness audit	At least one single-attribute fairness cut (approval rate, the four-fifths adverse impact ratio, and a false-negative/false-positive rate comparison) and, wherever the data contains a second relevant characteristic, one intersectional cut crossing two attributes at once, Sections 1.3.5 and 1.4.2's method.	An approval-rate table and adverse impact ratio for at least one single-attribute cut, and, if attempted, one intersectional cut, with an explicit statement of whether either violates the four-fifths rule.
8. Governance write-up	A short memo addressing the three SR 11-7 pillars for this specific model: what conceptual soundness was checked, what ongoing monitoring (a population stability index or a re-audit cadence) is proposed, and what outcomes analysis would be needed before this model could be trusted with a real decision, Section 1.4.1's method.	A half-page governance memo naming a concrete monitoring metric, its planned check frequency, and who (which of Chapter 15's three lines of defense) would be responsible for acting on it.

Table 1.8: An eight-stage template for a capstone project, distilled from this chapter's own pipeline

Wulin.Suo@Queensu.ca

list of what the project did not check.

That last list is the one most often skipped, and it is the one that matters most. Section 1.4.2 showed concretely that an unchecked intersectional cut can hide a violation a report's own headline numbers give no hint of. A capstone report that does not name its own blind spots is not more trustworthy for having omitted them; it is merely less honest about the same limitation every model in this chapter has shared.

Two failure modes are worth naming explicitly, since they are the most common ways a project satisfies this template's letter while missing its point. The first is treating each stage as an isolated checkbox rather than letting later stages inform earlier ones: Section 1.3.2's finding that the plain logistic regression's safest decile was miscalibrated should change which model Stage 6 and Stage 7 are actually run on, not be reported and then quietly ignored in favor of whichever model had the best Stage 3 accuracy. The second is stopping the fairness audit (Stage 7) at the first cut that happens to pass, the mistake this chapter's own Section 1.3.5 would have made had Section 1.4.2 never been run at all; a reader whose single-attribute cut passes cleanly should treat that result as a reason to look harder for an intersectional disparity, not as a reason to stop looking.

1.4.4 Capstone Project Ideas

The pipeline built in this chapter generalizes directly to other publicly available financial datasets: clean, engineer features, split, bake off models, calibrate, explain, choose a threshold, audit for fairness. Each of the following is a genuine capstone project using a real, freely available dataset, several of them also distributed on Kaggle under the names given.

- **Credit card fraud detection.** The Kaggle/Worldline/ULB "Credit Card Fraud Detection" dataset (284,807 European card transactions, 492 confirmed frauds, features anonymized via PCA) is far more imbalanced than this chapter's 22% default rate, a genuinely severe 0.17% positive rate that makes Chapter 15's precision-recall framing, and this chapter's calibration and cost-sensitive threshold methods, considerably more consequential than they were here; a reader completing this project should expect the naive 0.5 threshold to be almost useless, and should find that Section 1.3.4's analytic rule $c_{FP}/(c_{FP} + c_{FN})$ still locates the cost-minimizing threshold only if the model's probabilities are genuinely calibrated, which is far harder to achieve and to check with 492 positives than with this chapter's 6,636.

- **Consumer lending at greater scale.** The Lending Club loan dataset (millions of originated peer-to-peer loans with real interest rates, loan grades, and repayment outcomes) extends this chapter's pipeline to a dataset large enough that model training time itself becomes a design constraint, and includes enough temporal history to test a model's stability across a full economic cycle, the "non-stationarity" concern Chapter 6 and Chapter 17 both raised only in the abstract.

- **An algorithmic trading backtest on real prices.** Historical daily prices for the S&P 500 constituents (freely available from sources such as Yahoo Finance or Stooq) let a reader rebuild Chapter 13's pairs-trading or Chapter 12's mean-variance framework on real, not fictional, return series, confronting transaction costs, survivorship bias (delisted constituents silently missing from many free datasets), and the gap between an in-sample backtest and genuine out-of-sample performance directly.

- **Financial sentiment at scale.** The Financial PhraseBank dataset (thousands of analyst-labeled sentences from financial news, by agreement level) or a Twitter financial-sentiment dataset let a reader rebuild Chapter 16's Loughran-McDonald scoring and Naive Bayes classifier on genuinely labeled data, and compare a hand-built lexicon approach against a fine-tuned transformer the way Chapter 16's chapter itself only described.

- **A second, independent fraud dataset for cross-dataset generalization.** The IEEE-CIS Fraud Detection dataset (a Kaggle competition dataset with several hundred anonymized features spanning transaction, identity, and device information) is large and high-dimensional enough to make Section 1.3.3's SHAP-based explanation genuinely necessary rather than merely illustrative, since no analyst could manually inspect several hundred features' relationships to a prediction the way this chapter's 25-feature model still just barely permits.

Wulin.Suo@Queensu.ca

- **A robo-advisor: automated, risk-based portfolio construction.** Using free historical price data for a diversified set of low-cost exchange-traded funds (the kind of instrument Chapter 3 introduced), a reader can rebuild the core of a robo-advisor, following Chapter 12's own worked treatment directly: a short risk-tolerance questionnaire mapped to a target asset allocation, the mean-variance optimizer (or its risk-parity variant) computing the actual portfolio weights on real rather than illustrative return data, periodic rebalancing back to those target weights as prices drift, and, for a genuinely realistic version, tax-loss harvesting logic and a fee-drag comparison against a comparable actively managed alternative, extending Chapter 12's stylized numbers to a real portfolio and, ideally, a second real case besides Schwab's to compare against. This project differs from every other one in this list in an instructive way: it is not a classification or regression problem at all, but an optimization and decision-rule problem, so Chapter 18's SHAP and permutation-importance tools do not directly apply, and the natural "explainability" question becomes instead whether the resulting portfolio weights are traceable back to the questionnaire's stated risk tolerance and Chapter 12's stated optimization objective, a different, allocation-based notion of interpretability worth contrasting explicitly with this chapter's classification-based one.
- **Derivative pricing and delta-hedging on real options data.** Historical options-chain data for a liquid underlying such as the S&P 500 ETF (available from sources including Yahoo Finance, CBOE historical data files, and options-focused Kaggle datasets) lets a reader confront Chapter 5's Black-Scholes-Merton and binomial models with actual market prices rather than an assumed volatility input: back out each option's implied volatility from its traded price, plot the resulting volatility smile or skew across strikes (a pattern Chapter 5's constant-volatility assumption cannot produce at all, making its own limitation directly visible), and then implement and backtest a discrete-time delta-hedging strategy, rebalancing a hedge position at fixed intervals and tracking the resulting hedging error against the theoretical, continuous-rebalancing ideal Chapter 5 derives. This project is a natural pair with Chapter 14's market-microstructure material, since a real hedging desk's rebalancing frequency is itself constrained by the transaction costs Chapter 14 modeled, so a reader can extend the backtest to show how hedging error and transaction cost trade off against each other as the rebalancing interval shrinks.
- **Portfolio risk management through a real historical crisis.** Real daily return data for a multi-asset portfolio (a mix of equity, bond, and commodity ETFs, again freely available from sources such as Yahoo Finance or Stooq) lets a reader rebuild Chapter 8's Value-at-Risk and Expected Shortfall machinery, parametric, historical-simulation, and Monte Carlo, on genuine returns rather than a small illustrative covariance matrix, and then backtest the resulting VaR estimates using Chapter 8's own exception-counting method across a window that spans a real historical stress period, the 2008 financial crisis or the 2020 pandemic crash, both of which are included in most free multi-year daily price histories. The point of choosing a window with a real crisis in it deliberately is to test the assumption every one of Chapter 8's VaR methods quietly makes, that recent historical volatility and correlation are a reasonable guide to near-term risk, against a period where that assumption visibly failed, correlations across asset classes that looked low in calm markets rose sharply during the crisis itself, a phenomenon a reader can verify directly by comparing the portfolio's rolling covariance matrix estimated inside versus outside the crisis window, before feeding either estimate into Chapter 12's mean-variance or risk-parity optimizer and comparing the resulting portfolios' actual, realized risk during the crisis itself.
- **A smaller-scale replication for a first project.** The Statlog German Credit dataset (1,000 applicants, 20 features), also mirrored on Kaggle as "German Credit Risk," is small enough to rerun this entire chapter's pipeline, cleaning, feature engineering, bake-off, calibration, SHAP, threshold choice, fairness audit, in an afternoon, making it a natural first capstone for a reader who wants to practice the full workflow once, end to end, before tackling one of the larger datasets above.

1.5 Discussion Questions

1. Table 1.2 shows the class-weighted logistic regression achieving the best F_1 score among the two logistic models but the worst Brier score of all four models. If you were advising a lender that needed to report

Wulin.Suo@Queensu.ca

a single default probability to a regulator for every applicant, which of the four models would you recommend, and would your answer change if the lender only needed to rank applicants from safest to riskiest rather than report a specific probability to anyone?

2. Section 1.2.1 cleaned two undocumented category codes by folding them into an existing "other" category. Discuss an alternative cleaning strategy (for instance, treating each undocumented code as missing data and imputing it, or as its own separate category) and what evidence in the data would help you decide which approach is more defensible.

3. Section 1.3.3 showed permutation importance and SHAP disagreeing on whether `PAY_1` or `max_delinquency` is more important. If a regulator asked your institution which single feature drives most of your credit model's decisions, how would you respond given this disagreement, and does the question itself presuppose something that may not be true of the model?

4. Section 1.3.4's cost-minimizing threshold depends entirely on the assumed \$500-to-\$50 cost ratio. Propose a concrete way an institution could estimate these two costs from its own historical data, and discuss whether the same cost ratio should apply uniformly to every applicant or could reasonably vary by loan size.

5. Section 1.3.5's adverse impact ratio (0.960) passes the four-fifths rule comfortably, yet the same model refit with `SEX`, `MARRIAGE`, and `AGE` as inputs scored 0.843, and Section 1.4.2's sex-by-age cut fails (0.713). Discuss what additional analysis, beyond recomputing these same ratios next quarter, an internal model risk function should perform to decide whether the passing result reflects a stable, defensible model, and how much of the remaining disparity it should attribute to proxies for the omitted attributes rather than to genuine differences in default risk.

6. Choose one of Section 1.4.4's capstone project ideas and walk through Table 1.8's eight stages for it explicitly, one sentence per stage naming the specific dataset column, model choice, or metric that stage would use, and flag any stage whose deliverable that project's dataset cannot actually support (for instance, a dataset with no protected-characteristic field at all, which would leave Stage 7 unable to produce a real fairness cut).

7. Section 1.4.4's robo-advisor and portfolio-risk-management projects are not classification problems and have no natural confusion matrix, ROC curve, or SHAP value. For either one, propose a concrete way to check whether the resulting system is behaving as intended, given that this chapter's usual evaluation toolkit does not directly apply, and identify which of Table 1.8's eight stages would need to be reinterpreted rather than dropped entirely.

8. Section 1.4.2's intersectional cut checked sex crossed with age. Propose a third characteristic from this dataset worth crossing with sex in a real fair-lending examination, justify your choice, and discuss what evidence would tell you whether it is worth checking before you actually run the numbers.

9. Section 1.4.3 names two failure modes: treating each template stage as an isolated checkbox, and stopping a fairness audit at the first cut that happens to pass. Describe a plausible way a well-intentioned project team could fall into each failure mode without realizing it, and propose one concrete process change (not a new statistical method) that would catch each one before a project is reported as finished.

1.6 Challenges in End-to-End Financial AI Projects

Several challenges recur across this chapter's pipeline and connect directly to themes from earlier chapters.

Real data is never clean by default. Section 1.2.1's undocumented category codes have no equivalent in any earlier chapter's fictional dataset, and a project that skips this step, rather than merely inheriting clean data, inherits whatever silent errors those undocumented codes introduce into every downstream model.

The best model by one metric is not always the best model by every metric. Table 1.2's class-weighted logistic regression illustrates this directly: best F_1, worst Brier score, among the four models

Wulin.Suo@Queensu.ca

compared, so "best model" is only a well-defined question once the deployment use case fixes which metric actually matters.

Different explanation methods can disagree, and both can be right. Section 1.3.3's permutation-importance-versus-SHAP disagreement on the top two features is not a bug in either method, but a direct consequence of the two methods answering genuinely different questions, the caution Chapter 18 raised in the abstract and this chapter now confirms empirically.

A threshold choice is a business decision with real financial consequences, not a modeling detail. Section 1.3.4's more than twofold cost difference between the naive and cost-optimal thresholds dwarfs the performance gap between any two models in this chapter's bake-off, a genuine, quantified instance of Chapter 15's more abstract alert-economics argument.

Removing a protected attribute is not the same as removing its effect. Leaving `SEX`, `MARRIAGE`, and `AGE` out of the model, as fair-lending law requires, raised Section 1.3.5's sex-only adverse impact ratio from 0.843 to 0.960 at almost no cost in accuracy, yet the model still treats the sexes measurably differently and, never having seen `AGE`, still approves young men at well under four-fifths of the rate of older men, which is why Section 1.4.1 treated ongoing outcome monitoring, not the choice of inputs alone, as the real governance obligation.

A single-attribute fairness audit can pass while an intersectional one fails. Section 1.4.2's sex-and-age cut found a clear four-fifths-rule violation (0.713) that Section 1.3.5's sex-only audit (0.960, passing) gave no hint of, meaning an institution that stops testing after the first, cleaner-looking result may never discover a disparity that genuinely exists for a subgroup it never explicitly checked.

Not every financial AI project is a classification problem, and explainability means something different for each. Section 1.4.4's robo-advisor project has no confusion matrix, no ROC curve, and no natural SHAP value at all, since it is an optimization problem, not a supervised-learning one, so "explaining" its output means tracing a set of portfolio weights back to a stated risk tolerance and a stated optimization objective, a genuinely different task from anything Section 1.3.3's methods were built for, and a reminder that this book's explainability toolkit, powerful as it is for classification and regression, is not a universal solvent for every kind of financial AI system.

An end-to-end pipeline is only as trustworthy as its reproducibility. Every number in this chapter came from a fixed random seed and a documented sequence of cleaning, feature engineering, and modeling choices, recorded in the companion notebook precisely so that Section 1.4.1's independent-validation obligation is actually satisfiable rather than merely asserted.

1.7 Key Takeaways

This chapter's dataset, the UCI/Kaggle Default of Credit Card Clients data, was the first in this book not built to illustrate anything, and working through it surfaced problems earlier chapters' clean, fictional datasets never could: undocumented category and status codes and a misnamed column to clean (Section 1.2.1), a real class imbalance whose remedy (class weighting) traded a better F_1 score for a visibly worse Brier score (Section 1.3.1), and a calibration gap in the plain logistic regression's safest-looking decile that a rank-based evaluation alone would never have surfaced (Section 1.3.2).

Running Chapter 18's explanation toolkit with real software, `shap`'s `TreeExplainer` and scikit-learn's permutation importance, on a genuinely nonlinear gradient-boosted model showed both the same completeness property Chapter 18 verified by hand for a linear model and a real disagreement between two legitimate importance rankings (Section 1.3.3).

A concrete, assumption-driven cost-sensitive threshold choice cut expected cost by more than half relative to the naive 0.5 default (Section 1.3.4), though only at an operating point far stricter than any real lender would accept, an artifact of the toy cost ratio that the analytic threshold $c_{FP}/(c_{FP} + c_{FN})$ makes transparent. The resulting model, built without `SEX`, `MARRIAGE`, or `AGE` as inputs, passed a sex-only fairness audit comfortably but still showed both a residual sex difference and the same approval-rate-versus-error-rate tension Chapter 18 demonstrated with synthetic data, making Section 1.4.1's point directly: leaving out a protected attribute is not fairness, and a single deployment-time check is not governance; ongoing monitoring is.

Section 1.4.2 pushed the pipeline one level deeper still, in two directions. A systematic hyperparameter

Wulin.Suo@Queensu.ca

search found no consistent gain at all over Section 1.3.1's single, reasonably chosen XGBoost configuration, a useful and unglamorous reminder that model-family choice and feature engineering matter far more than fine-tuning once a sensible default configuration is already in place. An intersectional cut by sex and age then uncovered a stark, four-fifths-rule-violating disparity for young male applicants that Section 1.3.5's own single-attribute audit, run first and reported as passing, gave no indication of whatsoever. That contrast is the sharpest illustration in this entire book of why a fairness audit's scope, and not merely its arithmetic, determines what it can find.

It is worth closing on how much of this book's twenty preceding chapters this one pipeline actually drew on, since that breadth is itself the point of a capstone chapter. Chapter 2's pandas and NumPy fundamentals ran every line of this chapter's code. Chapter 6's train/test discipline, its feature engineering, its bagging and boosting concepts, and its overfitting caution structured Sections 1.2.2 and 1.3.1 directly. Chapter 9's calibration caveat, stated once in passing there, became Section 1.3.2's central finding. Chapter 15's class-imbalance remedies and cost-sensitive threshold framing became Sections 1.3.1 and 1.3.4's empirical results. And Chapter 18's entire explainability and fairness toolkit, adapted from hand computation on a linear model to real software on a genuine ensemble, became Sections 1.3.3, 1.3.5, and 1.4.2.

A single real dataset, considerably messier and less cooperative than any this book constructed on purpose, was enough to put nearly every technical tool this book developed to genuine, simultaneous use. That is the case this closing chapter exists to make. None of these methods was ever meant to be practiced in isolation, and a real financial AI project rarely affords the luxury of using only one at a time.

Section 1.4.3 distilled that same pipeline into an explicit, eight-stage template: problem framing, data cleaning and feature engineering, model bake-off, calibration, explainability, threshold and cost analysis, fairness audit, and a governance write-up. Each stage names its own required deliverable, precisely so that a reader is never left guessing what "finished" means for a project like this one.

The chapter, and the book, close with Section 1.4.4's further capstone project ideas. Each is a real, freely available dataset, spanning fraud detection, consumer lending, algorithmic trading, financial sentiment, robo-advisory, derivatives hedging, and credit scoring at a different scale. They are meant for a reader now ready to run this same disciplined pipeline independently, on data this book did not hand-pick: clean, engineer, split, compare, calibrate, explain, price the threshold, and audit for fairness.

1.8 Suggested Exercises

1. Using Section 1.2.1's method, tabulate the default rate separately for accounts with an undocumented `EDUCATION` code (0, 5, or 6) before cleaning, and compare it to the default rate for documented codes 1 through 4, discussing whether folding the undocumented codes into category 4 seems reasonable given what you find.

2. Using Table 1.2, compute each model's precision-recall trade-off directly (precision divided by recall) and rank the four models by this ratio, then explain in your own words what a very high or very low ratio implies about a model's operating point.

3. Using Section 1.3.2's method, compute the calibration gap (mean predicted probability minus mean actual default rate) for every decile of the random forest's predictions, and state whether the random forest's calibration more closely resembles the plain logistic regression's or XGBoost's in Table 1.3.

4. Section 1.2.2 kept `AGE` out of the model. Refit the XGBoost model with `AGE` added back as a 26th feature, use Section 1.3.3's method to compute its mean absolute SHAP value and its rank among the 26 features, and rerun Table 1.7's audit. Discuss, in light of Chapter 18's discussion of proxy features correlated with protected characteristics, whether leaving `AGE` out actually removed age-related information from the model's decisions.

5. Using Section 1.3.4's method, recompute the cost-minimizing threshold if the cost of a missed default is revised from \$500 to \$1,000 (holding the \$50 false-positive cost fixed), and explain in words the direction you expect the optimal threshold to move before recomputing it.

Wulin.Suo@Queensu.ca

6. Using Section 1.3.5's method, recompute the adverse impact ratio and the false-negative-rate gap at the naive 0.5 threshold instead of the cost-minimizing 0.10 threshold, and discuss whether the choice of threshold changes which group appears more favored by each of the two statistics.

7. Using Section 1.4.4's description of the Kaggle credit card fraud dataset, explain why a 0.17% positive rate makes accuracy an even less useful headline metric than it was for this chapter's 22.1% default rate, referencing Chapter 15's original caution about accuracy under class imbalance.

8. Using Table 1.7's method, compute the adverse impact ratio for a third cut, sex crossed with `EDUCATION` (collapsed to graduate school versus all other categories), and state whether this cut reveals a four-fifths-rule violation as stark as the sex-and-age cut in the text.

9. Section 1.4.2 found no consistent improvement from hyperparameter tuning. Propose one plausible reason a much larger or higher-dimensional dataset (such as the IEEE-CIS fraud dataset in Section 1.4.4) might show a larger gain from the same kind of search, referencing the bias-variance discussion in Chapter 6.

10. Using Section 1.3.2's isotonic recalibration result, explain in your own words why the recalibrated logistic regression's ROC-AUC barely changed (0.720 to 0.724) even though its Brier score improved measurably, referencing the monotonicity property isotonic regression is constrained to satisfy.

11. Explain why fitting an isotonic recalibration function directly on the test set, rather than via cross-validation on the training set alone, would invalidate the resulting calibration check, referencing Section 1.2.2's train/test discipline.

1.9 Further Reading

- Yeh and Lien, "The Comparisons of Data Mining Techniques for the Predictive Accuracy of Probability of Default of Credit Card Clients," *Expert Systems with Applications*. The original paper introducing the dataset this chapter's case study is built on.
- UCI Machine Learning Repository, "Default of Credit Card Clients Data Set." The public hosting page for the dataset used throughout this chapter, including its full data dictionary.
- Chen and Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. The original paper describing the gradient-boosting implementation used in Section 1.3.1.
- Lundberg and Lee, "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems*. The SHAP paper underlying the `TreeExplainer` algorithm used in Section 1.3.3, previously cited in Chapter 18 for its linear-model application.
- Niculescu-Mizil and Caruana, "Predicting Good Probabilities with Supervised Learning," *Proceedings of the 22nd International Conference on Machine Learning*. A foundational study of which model families tend to produce well-calibrated probabilities, directly relevant to Section 1.3.2's findings.
- Provost and Fawcett, *Data Science for Business*. A practitioner-oriented treatment of the cost-sensitive threshold reasoning developed in Section 1.3.4.
- Dua and Graff (curators), "UCI Machine Learning Repository," University of California, Irvine. The broader repository this chapter's dataset is drawn from, and a starting point for several of Section 1.4.4's further project ideas.
- Buolamwini and Gebru, "Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification," *Proceedings of the Conference on Fairness, Accountability, and Transparency*. The landmark empirical demonstration that a system can pass single-attribute fairness checks while still showing a stark disparity for an intersectional subgroup, the same pattern Section 1.4.2 found directly in a lending context.

Wulin.Suo@Queensu.ca